

Project: Analyzing Customer Churn

Module 2 | Chapter 8 | Notebook 2

In this final project you will analyze the customer churn of a telecommunications company. In this project you will do the following by yourself:

- Clean the data
 - Identify targets for marketing campaigns
 - Create visualizations to back up your findings
-

In this notebook you will be provided with very little instruction and structure. Try to get as far as you can. In the next chapter you will have access to the notebook [Project Assistance: Analyzing Customer Churn \(Chapter 9\)](#). That notebook contains the same project, but with additional instruction. If you have any problems, you can access the forum as usual and you can contact StackFuel Support.

Scenario: You work for a telecommunications company called Teleconfia. They are in the process of establishing themselves in the USA. They started by running a trial in Florida. New customers were able to use cellphones on the Teleconfia network at very low rates for one year. Not all customers stayed with Teleconfia for the whole year. Many of them left. This is known as customer churn. The company wants to tackle the problem of customer churn with two marketing campaigns.

The first marketing campaign is aimed at cities and will involve large posters. One city counts as a small town or a district in a larger city. The four cities with the highest rate of customer churn will be selected for the marketing campaign. The second marketing campaign will focus on individual customers. Customers who are likely to leave Teleconfia should receive a phone call to offer them a special deal.

The aim is to identify cities and customers who should be targeted with marketing campaigns. You will need to find out which data series you should use to make these decisions. You also have to find out if there are certain points from which customers should be contacted. Finally, use logistic regression to determine the critical value at which a customer is more likely to churn than remain.

You will have to justify your decisions and recommendations and visualize them.

You should split the project up into the following steps:

- **1) Import** the data

- **2) Check and clean** the data
- **3)** What are the names of the **four cities** with the highest rates of customer churn?
- **4a)** Which **categorical** data series should be used to identify customers who will possibly leave soon? Which customers should be contacted based on this data series?
- **4b)** Which **integer** (`int`) data series should be used to identify customers who might leave soon? How would you set the **threshold**? Which customers should be contacted based on this data series?
- **4c)** Which **floating point** (`float`) data series could you use to help with this selection? Determine the **threshold** for this using **logistic regression**. Which customers should be contacted based on this data series?
- **5)** Create **visualization** of the cities and the three other selected data series
- **6)** Formulate a recommendation.

We recommend that you stick to the order of the tasks above.

You have a lot of space to experiment in this notebook. The steps listed above are only given below as subheadings. If you need additional code cells, you can add new ones by clicking on the + button in the top bar, next to the save button (see [A First Look at Python \(Module 1, Chapter 1\)](#)).

You can access the data in a database named *telco_churn.db*. The following tables contain explanations about the database structure.

churn_data table:

Column number	Column name	Type	Description
0	account_length	numerical (<code>int</code>)	Unknown units of time, how long the customer has been a customer.
1	international_plan	categorical (nominal)	Contract with special conditions for cheaper calls to other countries.
2	voice_mail_plan	categorical (nominal)	Contract with special conditions for more voicemail storage.
3	number_vmail_messages	numerical (<code>int</code>)	Number of voicemail messages.
4	total_day_minutes	numerical (<code>float</code>)	Duration in minutes of all calls from 8am to 4pm.
5	total_day_calls	numerical (<code>int</code>)	Number of all calls from 8am to 4pm.

Column number	Column name	Type	Description
6	total_day_charge	numerical (float)	Calculated costs for all calls from 8am to 4pm.
7	total_eve_minutes	numerical (float)	Duration in minutes of all calls from 4pm to 10pm.
8	total_eve_calls	numerical (int)	Number of all calls from 4pm to 10pm.
9	total_eve_charge	numerical (float)	Calculated costs for all calls from 4pm to 10pm.
10	total_night_minutes	numerical (float)	Duration in minutes of all calls from 10pm to 8am.
11	total_night_calls	numerical (int)	Number of all calls from 10pm to 8am.
12	total_night_charge	numerical (float)	Calculated costs for all calls from 10pm to 8am.
13	customer_service_calls	numerical (int)	Number of calls to customer service, e.g. due to technical problems.
14	churn	categorical (nominal)	Did the customer leave? (1=yes 0=no)
15	local_area_code	categorical (nominal)	local area code for telephone.
16	phone_num	Categorical (nominal)	Customers telephone number not including the local area code.

cities table:

Column number	Column name	Type	Description
0	city	categorical (nominal)	Cities.
1	area_code	categorical (nominal)	local area code for telephone.

Congratulations: You have gone through the task description. Now you can work your way through the data pipeline as you have learned and practiced throughout this module: importing, cleaning, exploring, analyzing and visualizing the data. Now you will need to use the SQL skills you learned in Chapter 4.

1) Importing the data

For this step you will need to use things you learned in the following lessons:

- [Preparing Data with pandas](#) (Chapter 1)
- [Querying a Database](#) (Chapter 4)

- *Combining Two Tables* (Chapter 4)

```
In [1]: import pandas as pd
import sqlite3
```

```
In [2]: conn = sqlite3.connect('telco_churn.db')
```

```
In [3]: tables = conn.execute(
    "SELECT name FROM sqlite_master WHERE type='table';"
)

for table in tables:
    print(table)
```

```
('churn_data',)
('cities',)
```

```
In [4]: query = "SELECT * FROM churn_data"

df = pd.read_sql(query, conn)

df.head()
```

```
Out[4]:
```

	account_length	international_plan	voice_mail_plan	number_vmail_messages	total_day_minutes	total_day_calls	total_day_charge	total_eve_minutes	total
0	131.0	no	no	0.0	187.9	110.0	31.94	200.5	
1	63.0	yes	yes	21.0	151.5	99.0	25.76	147.8	
2	13.0	no	no	0.0	303.2	133.0	51.54	170.5	
3	129.0	no	no	0.0	159.1	100.0	27.05	202.5	
4	120.0	no	yes	28.0	215.8	123.0	36.69	285.2	

```
In [5]: cities_df = pd.read_sql("SELECT * FROM cities", conn)
```

```
cities_df.head()
```

```
Out[5]:
```

	city	area_code
0	Orlando1	321
1	Orlando2	407
2	Miami1	305
3	Miami2	786
4	Jacksonville	904

```
In [6]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3349 entries, 0 to 3348
Data columns (total 17 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   account_length                       3333 non-null   float64
1   international_plan                   3333 non-null   object
2   voice_mail_plan                      3333 non-null   object
3   number_vmail_messages                3311 non-null   float64
4   total_day_minutes                    3333 non-null   float64
5   total_day_calls                      3322 non-null   float64
6   total_day_charge                     3333 non-null   float64
7   total_eve_minutes                    3333 non-null   float64
8   total_eve_calls                      3325 non-null   float64
9   total_eve_charge                     3333 non-null   float64
10  total_night_minutes                  3333 non-null   float64
11  total_night_calls                    3328 non-null   float64
12  total_night_charge                   3333 non-null   float64
13  customer_service_calls               3333 non-null   float64
14  churn                               3333 non-null   float64
15  local_area_code                      3333 non-null   float64
16  phone_num                           3333 non-null   float64
dtypes: float64(15), object(2)
memory usage: 444.9+ KB
```

In [7]: `df.isna().sum()`

Out[7]:

account_length	16
international_plan	16
voice_mail_plan	16
number_vmail_messages	38
total_day_minutes	16
total_day_calls	27
total_day_charge	16
total_eve_minutes	16
total_eve_calls	24
total_eve_charge	16
total_night_minutes	16
total_night_calls	21
total_night_charge	16
customer_service_calls	16
churn	16
local_area_code	16
phone_num	16
dtype:	int64

In [8]: `df.describe()`

Out[8]:

	account_length	number_vmail_messages	total_day_minutes	total_day_calls	total_day_charge	total_eve_minutes	total_eve_calls	total_eve_charge	total_night_minutes	total_night_calls	total_night_charge
count	3333.000000	3311.000000	3333.000000	3322.000000	3333.000000	3333.000000	3325.000000	3333.000000	3333.000000	3322.000000	3333.000000
mean	101.064806	8.152824	179.883168	100.407586	30.562307	200.980348	100.105564	17.083540	101.064806	100.407586	30.562307
std	39.822106	13.717793	54.858487	20.046875	9.259435	50.713844	19.903150	4.310668	39.822106	20.046875	9.259435
min	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	0.000000
25%	74.000000	0.000000	143.700000	87.000000	24.430000	166.600000	87.000000	14.160000	74.000000	87.000000	24.430000
50%	101.000000	0.000000	179.400000	101.000000	30.500000	201.400000	100.000000	17.120000	101.000000	101.000000	30.500000
75%	127.000000	20.000000	216.400000	114.000000	36.790000	235.300000	114.000000	20.000000	127.000000	114.000000	36.790000
max	243.000000	51.000000	448.600000	165.000000	59.640000	363.700000	170.000000	30.910000	243.000000	165.000000	59.640000

Congratulations: You imported the data. Now you can begin cleaning it. This is such a varied task that you will need to use skills you have learned from almost every chapter in the second module. Don't let the task overwhelm you. Proceed step by step, checking different aspects of the dataset.

2) Check and clean the data

For this step you will need to use things you learned in the following lessons:

- [Preparing Data with pandas](#) (Chapter 1)
- [Analyzing Data with pandas \(1\)](#) (Chapter 1)
- [Exploring Data](#) (Chapter 2)
- [Project: Beer Ratings \(Data Cleaning\)](#) (Chapter 3))
- [Boolean Masking](#) (Chapter 4)
- [Project: Share Prices \(Preparing Data\)](#) (Chapter 6)

```
In [9]: df.isna().sum()
```

```
Out[9]: account_length      16
international_plan      16
voice_mail_plan         16
number_vmail_messages   38
total_day_minutes       16
total_day_calls         27
total_day_charge        16
total_eve_minutes       16
total_eve_calls         24
total_eve_charge        16
total_night_minutes     16
total_night_calls       21
total_night_charge      16
customer_service_calls  16
churn                   16
local_area_code         16
phone_num               16
dtype: int64
```

```
In [10]: df.describe()
```

Out[10]:

	account_length	number_vmail_messages	total_day_minutes	total_day_calls	total_day_charge	total_eve_minutes	total_eve_calls	total_eve_charge	total
count	3333.000000	3311.000000	3333.000000	3322.000000	3333.000000	3333.000000	3325.000000	3333.000000	
mean	101.064806	8.152824	179.883168	100.407586	30.562307	200.980348	100.105564	17.083540	
std	39.822106	13.717793	54.858487	20.046875	9.259435	50.713844	19.903150	4.310668	
min	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	74.000000	0.000000	143.700000	87.000000	24.430000	166.600000	87.000000	14.160000	
50%	101.000000	0.000000	179.400000	101.000000	30.500000	201.400000	100.000000	17.120000	
75%	127.000000	20.000000	216.400000	114.000000	36.790000	235.300000	114.000000	20.000000	
max	243.000000	51.000000	448.600000	165.000000	59.640000	363.700000	170.000000	30.910000	

In [11]:

```
negative_calls = df[df['customer_service_calls'] < 0]
print(negative_calls)
```

	account_length	international_plan	voice_mail_plan	\
55	101.0	no	yes	
173	110.0	no	yes	
230	64.0	no	no	
378	148.0	no	no	
659	103.0	no	yes	
698	183.0	no	yes	
715	90.0	no	yes	
832	159.0	no	yes	
851	58.0	no	yes	
876	66.0	no	no	
926	91.0	yes	no	
1046	121.0	no	no	
1116	184.0	yes	no	
1154	145.0	no	no	
1323	49.0	no	no	
1349	141.0	no	yes	
1469	90.0	no	no	
1591	120.0	yes	no	
1600	164.0	no	no	

1604	77.0	no	no
1718	93.0	yes	yes
1841	53.0	no	no
1957	156.0	no	no
2213	111.0	no	no
2263	110.0	no	no
2271	38.0	no	yes
2327	133.0	no	no
2369	126.0	yes	no
2380	65.0	no	no
2636	113.0	no	yes
2804	102.0	no	no
2869	79.0	no	no
2911	144.0	no	yes
2915	78.0	no	no
2975	113.0	no	no
3048	58.0	no	no
3283	110.0	yes	yes

	number_vmail_messages	total_day_minutes	total_day_calls	\
55	16.0	118.9	112.0	
173	38.0	236.3	102.0	
230	0.0	168.0	116.0	
378	0.0	208.4	120.0	
659	34.0	138.8	80.0	
698	8.0	86.5	119.0	
715	39.0	94.8	89.0	
832	23.0	153.6	93.0	
851	20.0	194.5	110.0	
876	0.0	167.3	91.0	
926	0.0	145.0	89.0	
1046	0.0	168.6	121.0	
1116	0.0	213.8	105.0	
1154	0.0	187.9	110.0	
1323	0.0	119.4	69.0	
1349	28.0	308.0	123.0	
1469	0.0	142.4	126.0	
1591	0.0	179.9	72.0	
1600	0.0	123.3	78.0	
1604	0.0	233.8	104.0	
1718	38.0	225.7	117.0	
1841	0.0	145.1	116.0	
1957	0.0	178.8	94.0	
2213	0.0	214.3	118.0	

2263	0.0	222.7	94.0
2271	25.0	142.4	106.0
2327	0.0	176.8	92.0
2369	0.0	197.6	126.0
2380	0.0	148.7	80.0
2636	32.0	180.4	89.0
2804	0.0	129.5	56.0
2869	0.0	205.7	123.0
2911	48.0	189.8	96.0
2915	0.0	87.7	74.0
2975	0.0	239.7	47.0
3048	0.0	149.4	145.0
3283	27.0	267.9	103.0

	total_day_charge	total_eve_minutes	total_eve_calls	total_eve_charge	\
55	20.21	228.3	97.0	19.41	
173	40.17	195.9	112.0	16.65	
230	28.56	192.4	94.0	16.35	
378	35.43	174.4	99.0	14.82	
659	23.60	142.0	108.0	12.07	
698	14.71	285.2	97.0	24.24	
715	16.12	219.1	91.0	18.62	
832	26.11	216.9	88.0	18.44	
851	33.07	213.7	89.0	18.16	
876	28.44	230.0	68.0	19.55	
926	24.65	175.8	102.0	14.94	
1046	28.66	168.6	94.0	14.33	
1116	36.35	159.6	84.0	13.57	
1154	31.94	197.0	117.0	16.75	
1323	20.30	273.3	92.0	23.23	
1349	52.36	247.8	128.0	21.06	
1469	24.21	126.2	118.0	10.73	
1591	30.58	170.0	98.0	14.45	
1600	20.96	170.0	85.0	14.45	
1604	39.75	266.5	94.0	22.65	
1718	38.37	119.6	122.0	10.17	
1841	24.67	233.7	82.0	19.86	
1957	30.40	178.4	97.0	15.16	
2213	36.43	208.5	76.0	17.72	
2263	37.86	105.8	98.0	8.99	
2271	24.21	313.7	109.0	26.66	
2327	30.06	187.5	97.0	15.94	
2369	33.59	246.5	112.0	20.95	
2380	25.28	259.0	94.0	22.02	

2636	30.67	129.4	124.0	11.00
2804	22.02	354.2	118.0	30.11
2869	34.97	214.5	108.0	18.23
2911	32.27	123.4	67.0	10.49
2915	14.91	214.8	58.0	18.26
2975	40.75	282.9	110.0	24.05
3048	25.40	196.5	105.0	16.70
3283	45.54	263.3	74.0	22.38

	total_night_minutes	total_night_calls	total_night_charge	\
55	180.1	111.0	8.10	
173	183.5	82.0	8.26	
230	166.5	98.0	7.49	
378	310.7	105.0	13.98	
659	183.8	77.0	8.27	
698	180.4	133.0	8.12	
715	197.4	65.0	8.88	
832	161.3	91.0	7.26	
851	236.6	92.0	10.65	
876	191.7	118.0	8.63	
926	223.7	151.0	10.07	
1046	95.3	59.0	4.29	
1116	139.2	137.0	6.26	
1154	167.0	108.0	7.52	
1323	214.4	153.0	9.65	
1349	152.9	103.0	6.88	
1469	274.2	71.0	12.34	
1591	190.6	89.0	8.58	
1600	165.9	78.0	7.47	
1604	212.7	104.0	9.57	
1718	193.2	125.0	8.69	
1841	208.7	95.0	9.39	
1957	169.2	77.0	7.61	
2213	182.4	98.0	8.21	
2263	214.8	78.0	9.67	
2271	126.6	117.0	5.70	
2327	196.8	88.0	8.86	
2369	285.3	104.0	12.84	
2380	149.5	107.0	6.73	
2636	166.9	124.0	7.51	
2804	145.5	93.0	6.55	
2869	226.1	106.0	10.17	
2911	214.2	106.0	9.64	
2915	201.3	147.0	9.06	

2975	238.4	88.0	10.73
3048	209.5	108.0	9.43
3283	178.1	106.0	8.01

	customer_service_calls	churn	local_area_code	phone_num
55	-2.0	0.0	941.0	2787.0
173	-1.0	0.0	305.0	8845.0
230	-2.0	0.0	305.0	8518.0
378	-1.0	0.0	850.0	7464.0
659	-1.0	0.0	239.0	5053.0
698	-2.0	0.0	813.0	8890.0
715	-1.0	0.0	786.0	3931.0
832	-2.0	0.0	561.0	9825.0
851	-1.0	0.0	239.0	1259.0
876	-1.0	1.0	407.0	5155.0
926	-2.0	1.0	786.0	7754.0
1046	-1.0	0.0	850.0	2458.0
1116	-2.0	0.0	813.0	6381.0
1154	-2.0	0.0	239.0	4288.0
1323	-2.0	0.0	239.0	4519.0
1349	-1.0	0.0	850.0	8583.0
1469	-1.0	0.0	727.0	5994.0
1591	-1.0	1.0	561.0	7611.0
1600	-1.0	0.0	727.0	1746.0
1604	-2.0	0.0	904.0	8814.0
1718	-1.0	1.0	904.0	5632.0
1841	-2.0	0.0	727.0	8023.0
1957	-1.0	0.0	850.0	6223.0
2213	-1.0	0.0	386.0	9997.0
2263	-1.0	0.0	386.0	4558.0
2271	-2.0	0.0	305.0	8953.0
2327	-2.0	0.0	239.0	8996.0
2369	-2.0	0.0	904.0	9798.0
2380	-2.0	0.0	786.0	5908.0
2636	-1.0	0.0	407.0	8112.0
2804	-1.0	0.0	305.0	9774.0
2869	-1.0	0.0	321.0	3515.0
2911	-2.0	1.0	407.0	3049.0
2915	-1.0	0.0	904.0	5721.0
2975	-2.0	1.0	813.0	4418.0
3048	-1.0	0.0	941.0	7822.0
3283	-1.0	1.0	904.0	8024.0

```
In [12]: df.loc[df['customer_service_calls'] < 0,
             'customer_service_calls'] = pd.NA
```

```
In [13]: df.isna().sum()
```

```
Out[13]: account_length      16
international_plan      16
voice_mail_plan         16
number_vmail_messages   38
total_day_minutes       16
total_day_calls         27
total_day_charge        16
total_eve_minutes       16
total_eve_calls         24
total_eve_charge        16
total_night_minutes     16
total_night_calls       21
total_night_charge      16
customer_service_calls  53
churn                   16
local_area_code         16
phone_num               16
dtype: int64
```

```
In [14]: df['customer_service_calls'] = (
            df['customer_service_calls']
            .fillna(df['customer_service_calls'].median())
        )
```

```
In [15]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3349 entries, 0 to 3348
Data columns (total 17 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   account_length      3333 non-null   float64
 1   international_plan   3333 non-null   object
 2   voice_mail_plan     3333 non-null   object
```

```

3  number_vmail_messages  3311 non-null  float64
4  total_day_minutes      3333 non-null  float64
5  total_day_calls        3322 non-null  float64
6  total_day_charge       3333 non-null  float64
7  total_eve_minutes      3333 non-null  float64
8  total_eve_calls        3325 non-null  float64
9  total_eve_charge       3333 non-null  float64
10 total_night_minutes    3333 non-null  float64
11 total_night_calls      3328 non-null  float64
12 total_night_charge     3333 non-null  float64
13 customer_service_calls 3349 non-null  float64
14 churn                  3333 non-null  float64
15 local_area_code        3333 non-null  float64
16 phone_num              3333 non-null  float64

```

```
dtypes: float64(15), object(2)
```

```
memory usage: 444.9+ KB
```

Missing Value and Data Type Strategy

Missing values were preserved where possible in order to avoid unnecessary customer deletion during bivariate analysis.

Implausible values (such as negative call counts) were converted to missing values (NaN) instead of removing complete customer records.

Median imputation was applied selectively for variables required in downstream analysis and modeling.

Integer-like variables and categorical variables were standardized to appropriate data types to improve consistency and analysis quality.

In [16]:

```

# Standardize column names

df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(' ', '_')
)

```

In [17]:

```

# Convert categorical variables

df['international_plan'] = (
    df['international_plan']

```

```
.astype('category')
)

df['voice_mail_plan'] = (
    df['voice_mail_plan']
    .astype('category')
)

# Convert churn to integer

df['churn'] = (
    pd.to_numeric(df['churn'], errors='coerce')
    .astype('Int64')
)
```

```
In [18]: int_like = [
    'customer_service_calls',
    'account_length',
    'number_vmail_messages',
    'total_day_calls',
    'total_eve_calls',
    'total_night_calls'
]
```

Congratulations: You have cleaned the data. For a lot of data projects, this can take up just as much time as analyzing the data. You can move onto that now.

3) What are the names of the four cities with the highest rates of customer churn?

For this step you will need to use things you learned in the following lessons:

- [Exploring Categories](#) (Chapter 2)
- [Boolean Masking](#) (Chapter 4)

```
In [19]: print(cities_df.columns)
```

```
Index(['city', 'area_code'], dtype='object')
```

```
In [20]: df = pd.merge(
    df,
    cities_df,
    left_on='local_area_code',
    right_on='area_code',
    how='left'
)
```

```
In [21]: df[['local_area_code', 'city']].head()
```

```
Out[21]:
```

	local_area_code	city
0	850.0	Tallahassee
1	786.0	Miami2
2	904.0	Jacksonville
3	305.0	Miami1
4	239.0	Cape Coral

```
In [22]: city_churn = (
    df.groupby('city')['churn']
    .mean()
    .sort_values(ascending=False)
)
```

```
In [23]: print("Top Cities by Churn Rate:")
print(city_churn.head(4))
```

```
Top Cities by Churn Rate:
city
Jacksonville    0.298193
Orlando1         0.236667
Cape Coral       0.217822
Orlando2         0.190789
Name: churn, dtype: float64
```

Congratulations: You have identified four cities where the marketing campaign should be launched. Next you will identify individual customers who should be contacted.

4a) Which **categorical** data series should be used to identify customers who will possibly leave soon? Which customers should be contacted based on this data series?

Base your analysis on the whole data set, taking into account customers in all cities. For this step, you will find things you learned in the following lessons helpful:

- [Exploring Data](#) (Chapter 2)
- [Exploring Categories](#) (Chapter 2)
- [Boolean Masking](#) (Chapter 4)

```
In [24]: df.dtypes
```

```
Out[24]: account_length      float64
international_plan    category
voice_mail_plan       category
number_vmail_messages float64
total_day_minutes     float64
total_day_calls       float64
total_day_charge      float64
total_eve_minutes     float64
total_eve_calls       float64
total_eve_charge      float64
total_night_minutes   float64
total_night_calls     float64
total_night_charge    float64
customer_service_calls float64
churn                 Int64
local_area_code       float64
phone_num             float64
city                  object
area_code             float64
dtype: object
```

In [25]:

```
categorical_vars = [  
    'international_plan',  
    'voice_mail_plan'  
]  
  
print(categorical_vars)
```

```
['international_plan', 'voice_mail_plan']
```

In [26]:

```
intl_churn = (  
    df.groupby('international_plan')['churn']  
    .mean()  
)  
  
print(intl_churn)
```

```
international_plan  
no      0.114950  
yes     0.424149  
Name: churn, dtype: float64
```

In [27]:

```
vm_churn = (  
    df.groupby('voice_mail_plan')['churn']  
    .mean()  
)  
  
print(vm_churn)
```

```
voice_mail_plan  
no      0.167151  
yes     0.086768  
Name: churn, dtype: float64
```

In [28]:

```
print("International Plan Churn Rates:")  
print(intl_churn)  
  
print("\nVoice Mail Plan Churn Rates:")  
print(vm_churn)
```

International Plan Churn Rates:

international_plan

no 0.114950

yes 0.424149

Name: churn, dtype: float64

Voice Mail Plan Churn Rates:

voice_mail_plan

no 0.167151

yes 0.086768

Name: churn, dtype: float64

In [29]:

```
high_risk_intl = df[
    (df['international_plan'] == 'yes') &
    (df['churn'] == 0)
]

print(high_risk_intl.head())
```

	account_length	international_plan	voice_mail_plan	number_vmail_messages	\
1	63.0	yes	yes	21.0	
24	91.0	yes	no	0.0	
33	61.0	yes	yes	29.0	
57	68.0	yes	yes	39.0	
59	12.0	yes	no	0.0	

	total_day_minutes	total_day_calls	total_day_charge	total_eve_minutes	\
1	151.5	99.0	25.76	147.8	
24	151.8	115.0	25.81	103.6	
33	128.2	119.0	21.79	171.7	
57	142.0	140.0	24.14	241.6	
59	216.7	117.0	36.84	116.5	

	total_eve_calls	total_eve_charge	total_night_minutes	total_night_calls	\
1	89.0	12.56	210.4	114.0	
24	116.0	8.81	156.3	86.0	
33	83.0	14.59	250.9	114.0	
57	89.0	20.54	302.0	72.0	
59	126.0	9.90	220.0	110.0	

	total_night_charge	customer_service_calls	churn	local_area_code	\
1	9.47	1.0	0	786.0	
24	7.03	1.0	0	561.0	

33	11.29	3.0	0	407.0
57	13.59	1.0	0	727.0
59	9.90	2.0	0	321.0

	phone_num	city	area_code
1	8029.0	Miami2	786.0
24	9383.0	West Palm Beach	561.0
33	3843.0	Orlando2	407.0
57	2138.0	Clearwater	727.0
59	5211.0	Orlando1	321.0

Congratulations: You have found a way to categorize the customers in relation to customer churn. Next you will focus on a numerical data series that also correlates with customer churn.

4b) Which integer data series would you also use for this and how would you set the threshold? Which customers should be contacted based on this data series?

You might find the following lessons helpful for this step:

- [Exploring Data \(Chapter 2\)](#)
- [Visualizing Data Distributions using Histograms \(Chapter 2\)](#)
- [Exploring Categories \(Chapter 2\)](#)
- [Importing and Cleaning a Business Data Set \(Chapter 2\)](#)
- [Box Plots \(Chapter 3\)](#)
- [Boolean Masking \(Chapter 4\)](#)

```
In [30]: print(int_like)

['customer_service_calls', 'account_length', 'number_vmail_messages', 'total_day_calls', 'total_eve_calls', 'total_night_calls']
```

```
In [31]: import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [32]: for col in int_like:

    print(f"\nVariable: {col}")
```

```
print(
    df.groupby('churn')[col]
    .median()
)

plt.figure(figsize=(8,5))

sns.boxplot(
    x='churn',
    y=col,
    data=df
)

plt.title(f'{col} vs Churn')

plt.show()
```

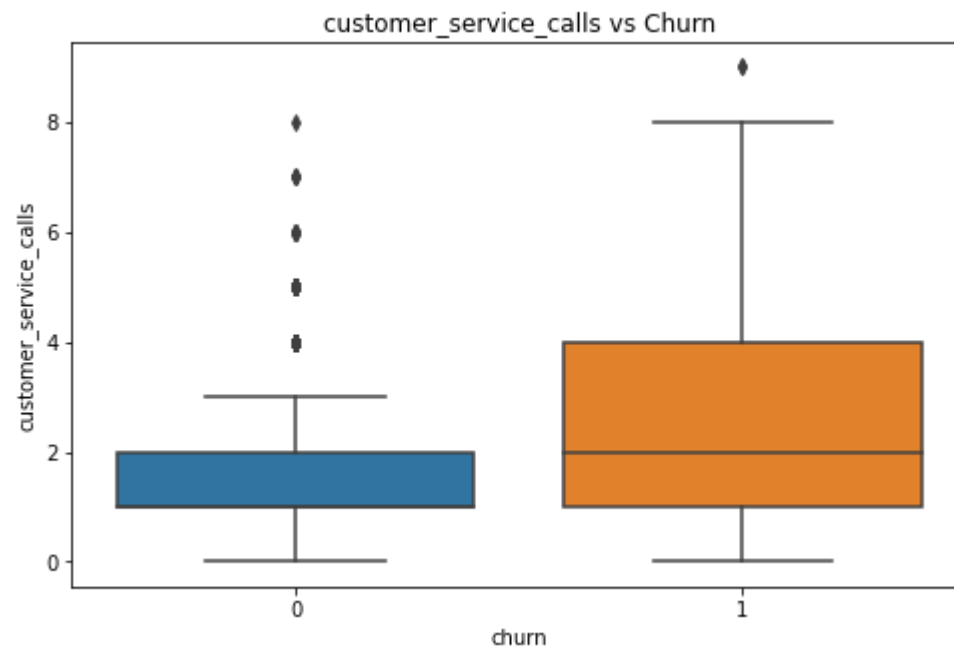
Variable: customer_service_calls

churn

0 1.0

1 2.0

Name: customer_service_calls, dtype: float64



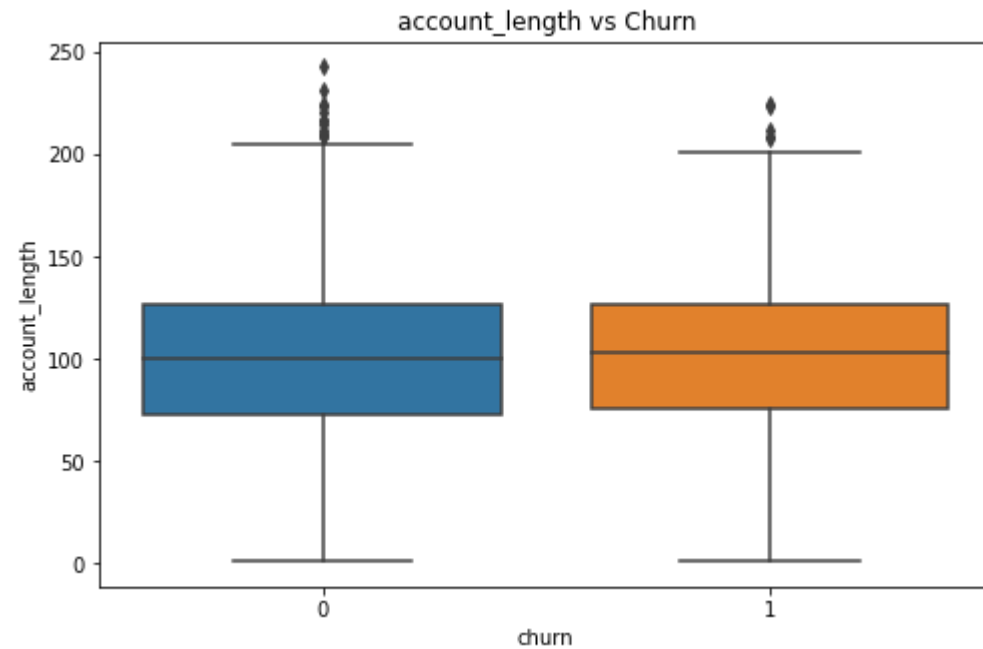
Variable: account_length

churn

0 100.0

1 103.0

Name: account_length, dtype: float64



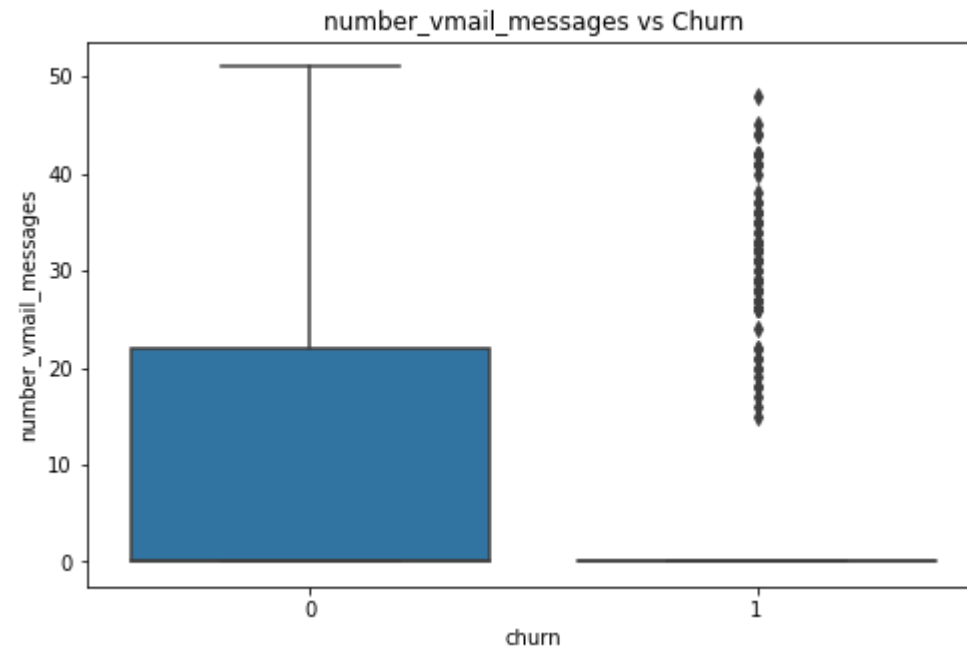
Variable: number_vmail_messages

churn

0 0.0

1 0.0

Name: number_vmail_messages, dtype: float64



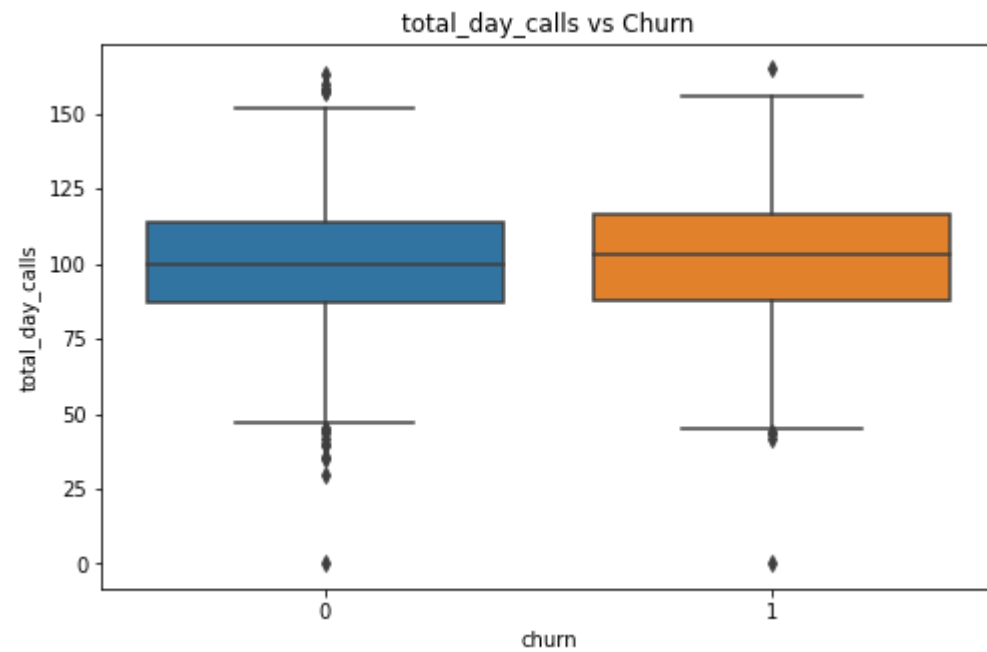
Variable: total_day_calls

churn

0 100.0

1 103.0

Name: total_day_calls, dtype: float64



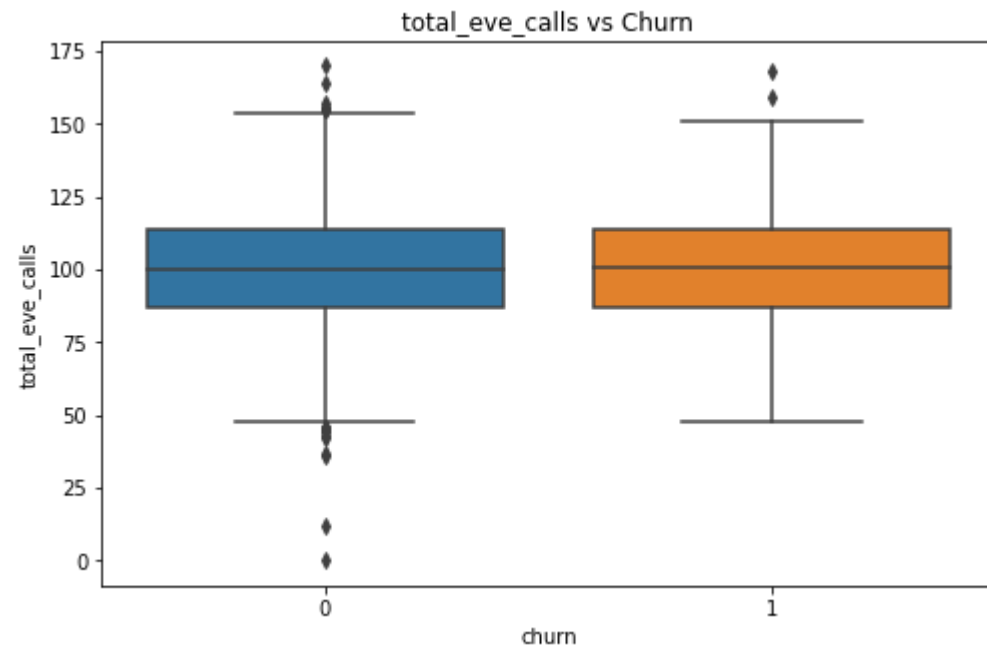
Variable: total_eve_calls

churn

0 100.0

1 101.0

Name: total_eve_calls, dtype: float64



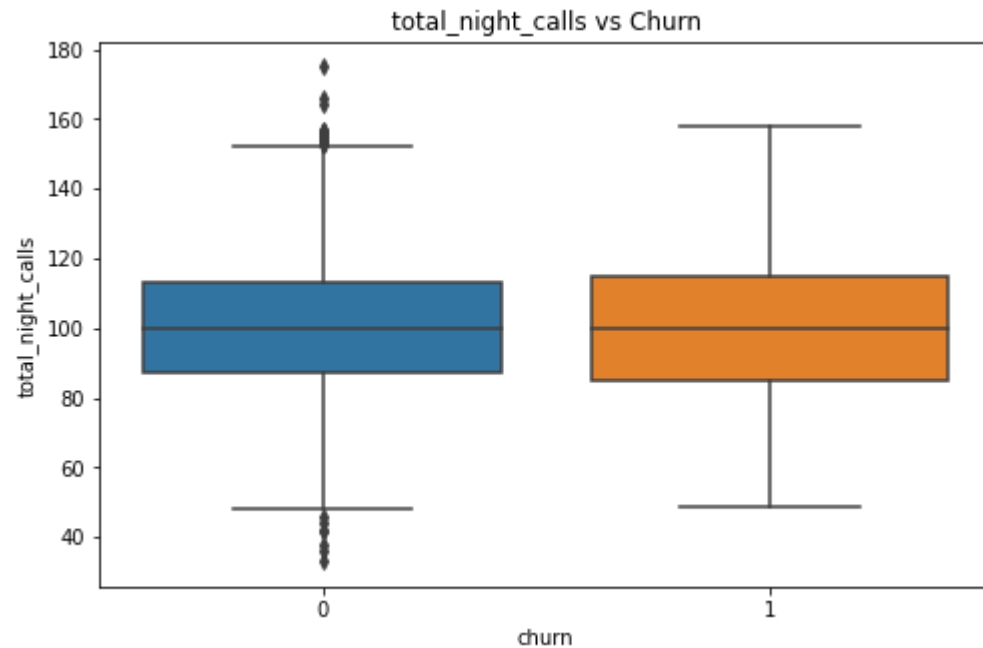
Variable: total_night_calls

churn

0 100.0

1 100.0

Name: total_night_calls, dtype: float64



Integer Variable Selection

After comparing all integer-like variables, customer_service_calls demonstrated the clearest relationship with churn.

Customers making more than three service calls showed substantially higher churn rates, making this variable highly useful for customer-retention targeting.

```
In [33]: import matplotlib.pyplot as plt
import seaborn as sns

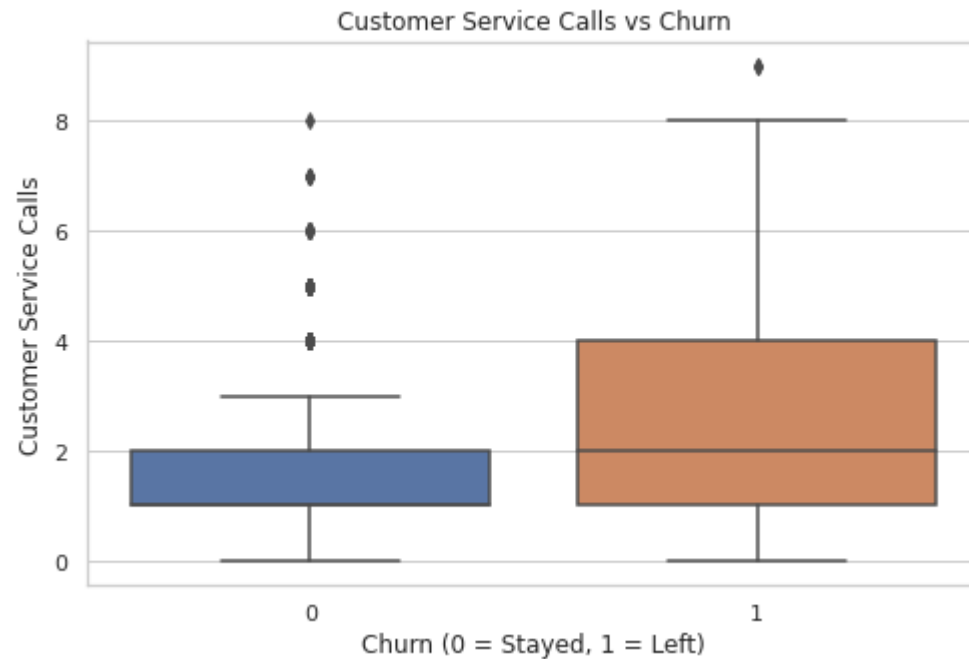
sns.set(style="whitegrid")

plt.figure(figsize=(8,5))

sns.boxplot(
    x='churn',
    y='customer_service_calls',
    data=df
)
```

```
plt.title('Customer Service Calls vs Churn')
plt.xlabel('Churn (0 = Stayed, 1 = Left)')
plt.ylabel('Customer Service Calls')

plt.show()
```



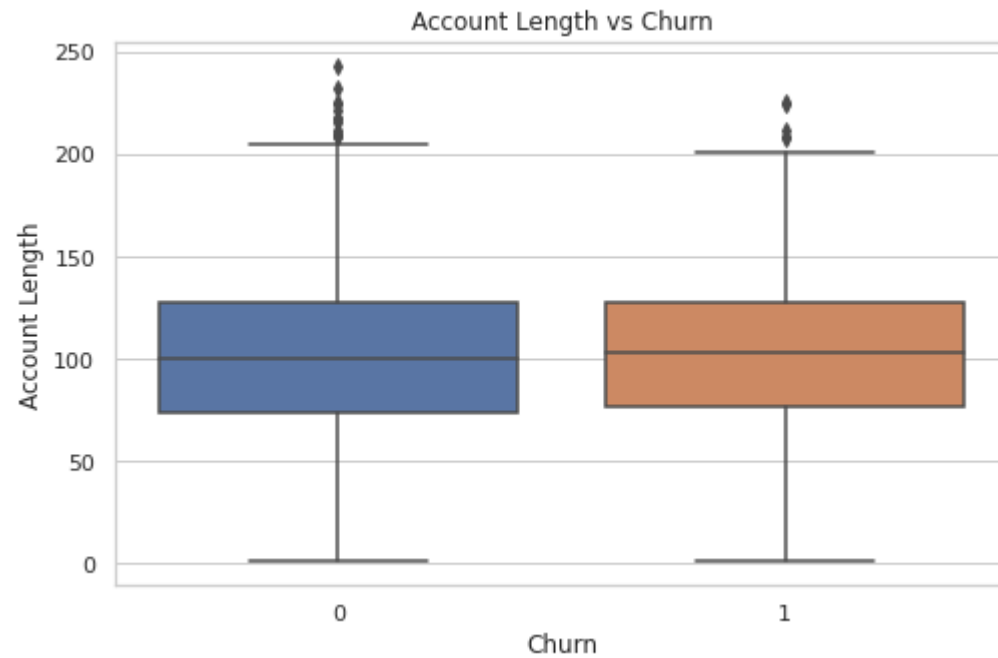
In [34]:

```
plt.figure(figsize=(8,5))

sns.boxplot(
    x='churn',
    y='account_length',
    data=df
)

plt.title('Account Length vs Churn')
plt.xlabel('Churn')
plt.ylabel('Account Length')

plt.show()
```



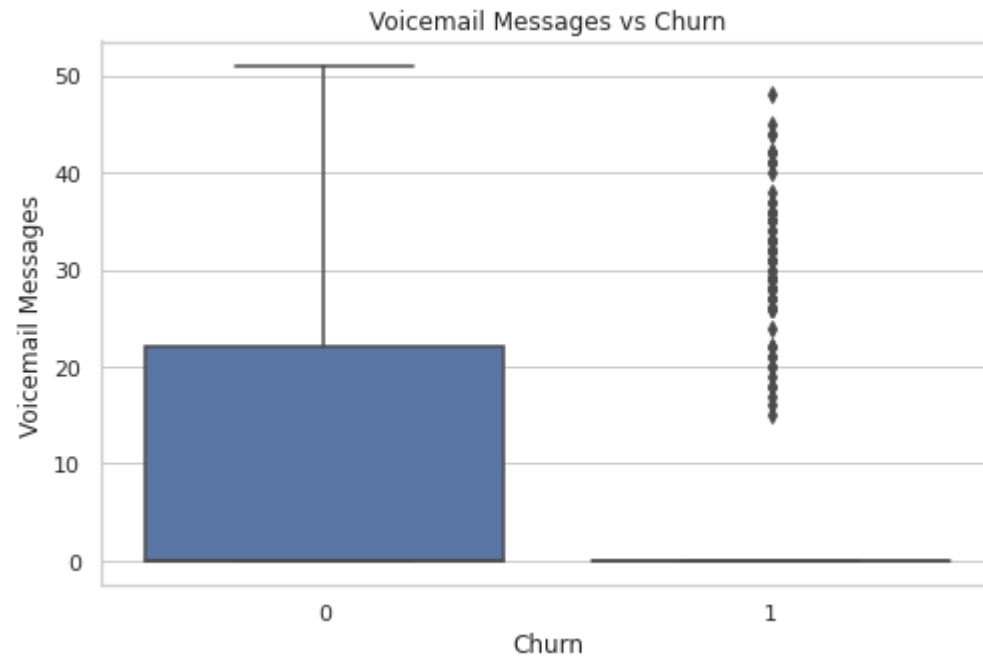
In [35]:

```
plt.figure(figsize=(8,5))

sns.boxplot(
    x='churn',
    y='number_vmail_messages',
    data=df
)

plt.title('Voicemail Messages vs Churn')
plt.xlabel('Churn')
plt.ylabel('Voicemail Messages')

plt.show()
```



In [36]:

```
service_call_churn = (  
    df.groupby('customer_service_calls')['churn']  
    .mean()  
)  
  
print(service_call_churn)
```

```
customer_service_calls  
0.0    0.131994  
1.0    0.104428  
2.0    0.113055  
3.0    0.102564  
4.0    0.457831  
5.0    0.606061  
6.0    0.636364  
7.0    0.555556  
8.0    0.500000  
9.0    1.000000  
Name: churn, dtype: float64
```

In [37]:

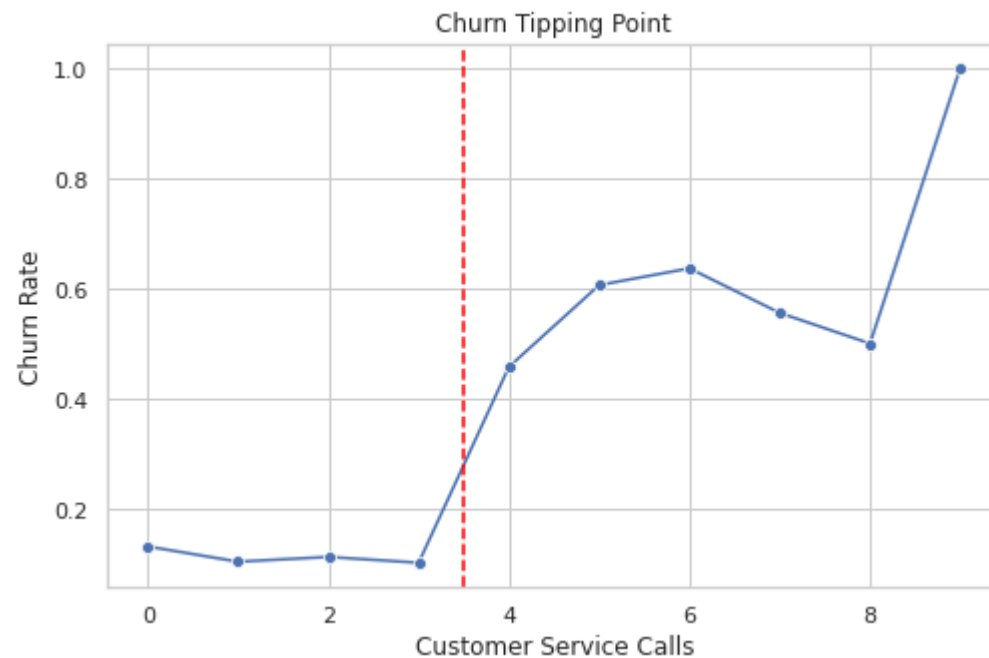
```
plt.figure(figsize=(8,5))

sns.lineplot(
    x=service_call_churn.index,
    y=service_call_churn.values,
    marker='o'
)

plt.axvline(x=3.5, color='red', linestyle='--')

plt.title('Churn Tipping Point')
plt.xlabel('Customer Service Calls')
plt.ylabel('Churn Rate')

plt.show()
```



In [38]:

```
high_risk_calls = df[
    (df['customer_service_calls'] > 3) &
    (df['churn'] == 0)
]
```

```
print(high_risk_calls.head())
```

	account_length	international_plan	voice_mail_plan	number_vmail_messages	\
7	107.0	no	no	0.0	
58	104.0	no	no	0.0	
87	101.0	no	no	0.0	
157	64.0	no	yes	40.0	
164	67.0	no	no	0.0	

	total_day_minutes	total_day_calls	total_day_charge	total_eve_minutes	\
7	189.7	76.0	32.25	156.1	
58	280.4	127.0	47.67	179.4	
87	153.8	89.0	26.15	234.0	
157	210.0	116.0	35.70	232.7	
164	260.4	107.0	44.27	208.2	

	total_eve_calls	total_eve_charge	total_night_minutes	\
7	65.0	13.27	244.0	
58	79.0	15.25	150.6	
87	89.0	19.89	196.3	
157	89.0	19.78	168.8	
164	104.0	17.70	207.9	

	total_night_calls	total_night_charge	customer_service_calls	churn	\
7	91.0	10.98	5.0	0	
58	77.0	6.78	5.0	0	
87	77.0	8.83	4.0	0	
157	94.0	7.60	8.0	0	
164	115.0	9.36	6.0	0	

	local_area_code	phone_num	city	area_code
7	407.0	1399.0	Orlando2	407.0
58	727.0	2026.0	Clearwater	727.0
87	850.0	2292.0	Tallahassee	850.0
157	786.0	8822.0	Miami2	786.0
164	813.0	5370.0	Tampa	813.0

Congratulations: You have identified an integer data series that you can use to identify customers who are more likely to leave. Next you need to identify a floating point data series that you can use to predict which customers are likely to leave.

4c) Which floating point data series could you use to help with this selection? Determine the threshold for this by using **logistic regression**. Which customers should be contacted based on this data series?

For this step you will need to use skills you learned in the following lessons:

- [Visualizing Data Distributions using Histograms](#) (Chapter 2)
- [Visualizing Correlations with Scatter Plots](#) (Chapter 2)
- [Box Plots](#) (Chapter 3)
- [Project: Share Prices Linear Regression Modeling](#) (Chapter 6)
- [Making Predictions Using Logistic Regression](#) (Chapter 3)
- [Boolean Masking](#) (Chapter 4)
- [Measures of Central Tendency](#) (Chapter 3)

In [39]:

```
float_candidates = [  
    'total_day_charge',  
    'total_eve_charge',  
    'total_night_charge',  
    'total_day_minutes'  
]  
  
print(float_candidates)
```

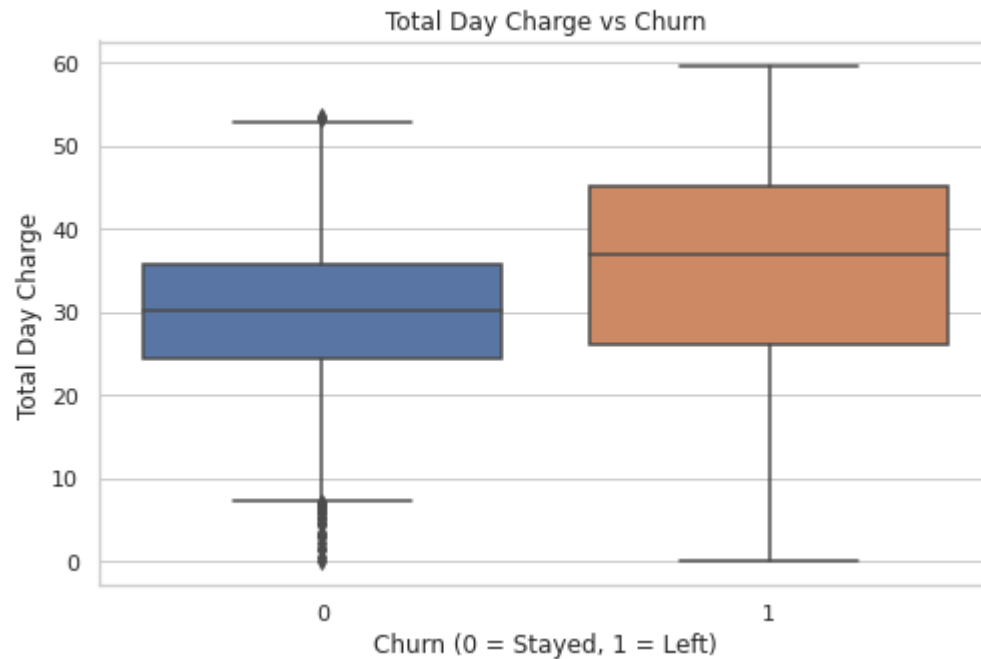
```
['total_day_charge', 'total_eve_charge', 'total_night_charge', 'total_day_minutes']
```

In [40]:

```
import matplotlib.pyplot as plt  
import seaborn as sns  
  
sns.set(style="whitegrid")  
  
plt.figure(figsize=(8,5))  
  
sns.boxplot(  
    x='churn',  
    y='total_day_charge',  
    data=df  
)
```

```
plt.title('Total Day Charge vs Churn')
plt.xlabel('Churn (0 = Stayed, 1 = Left)')
plt.ylabel('Total Day Charge')

plt.show()
```



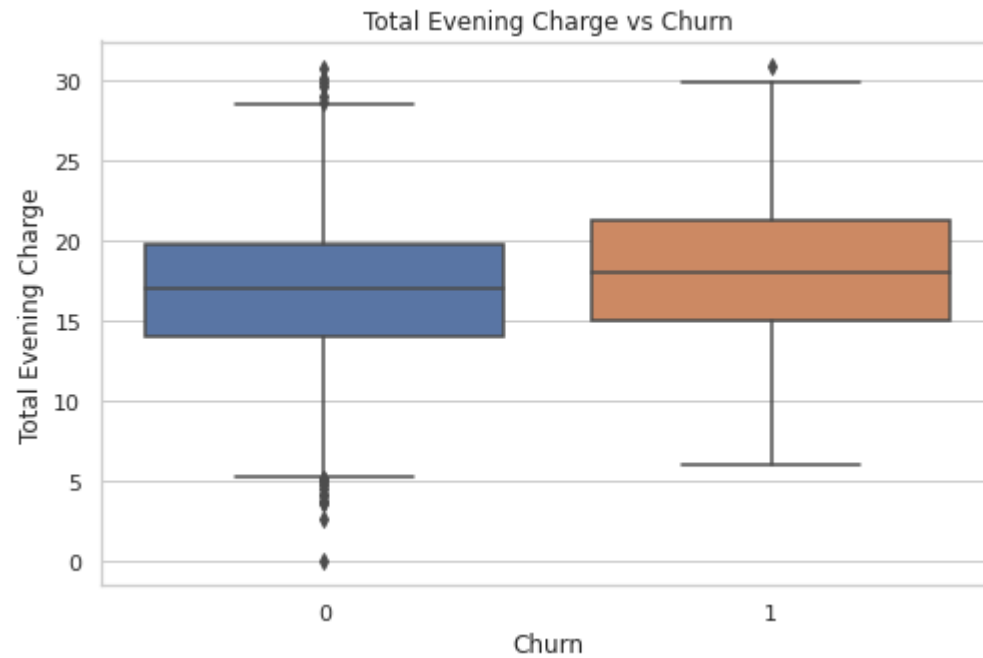
In [41]:

```
plt.figure(figsize=(8,5))

sns.boxplot(
    x='churn',
    y='total_eve_charge',
    data=df
)

plt.title('Total Evening Charge vs Churn')
plt.xlabel('Churn')
plt.ylabel('Total Evening Charge')

plt.show()
```



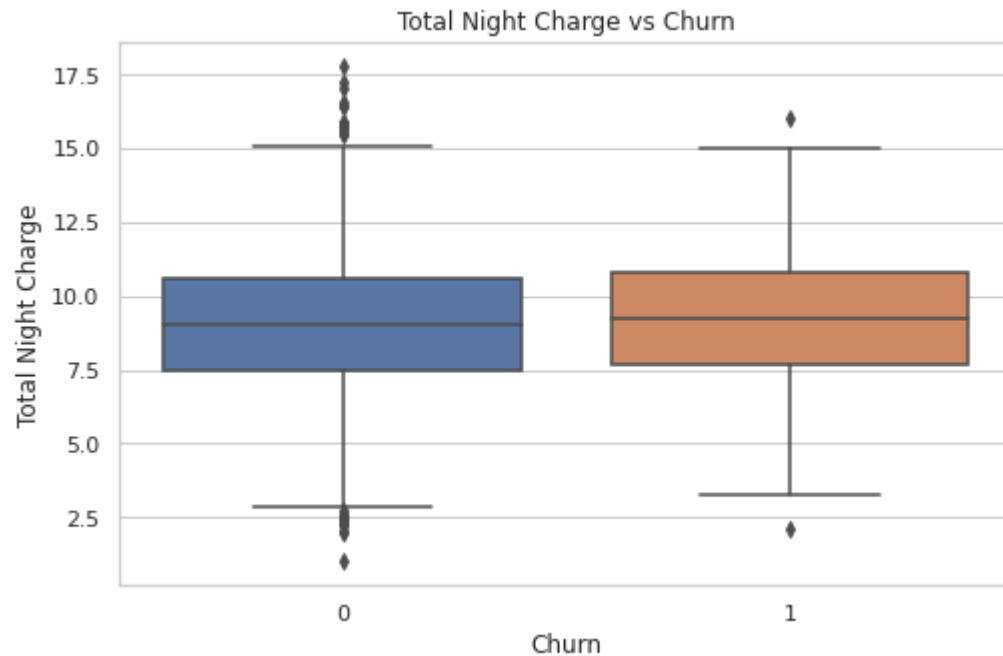
In [42]:

```
plt.figure(figsize=(8,5))

sns.boxplot(
    x='churn',
    y='total_night_charge',
    data=df
)

plt.title('Total Night Charge vs Churn')
plt.xlabel('Churn')
plt.ylabel('Total Night Charge')

plt.show()
```



```
In [43]: for col in float_candidates:
          print(f"\nVariable: {col}")
          print(
              df.groupby('churn')[col]
                .mean()
          )
```

Variable: total_day_charge
churn
0 29.780421
1 35.175921
Name: total_day_charge, dtype: float64

Variable: total_eve_charge
churn
0 16.918909
1 18.054969
Name: total_eve_charge, dtype: float64

```
Variable: total_night_charge
churn
0    9.006074
1    9.235528
Name: total_night_charge, dtype: float64
```

```
Variable: total_day_minutes
churn
0    175.226211
1    207.362112
Name: total_day_minutes, dtype: float64
```

Float Variable Selection

After comparing all major float variables, total_day_charge demonstrated the clearest separation between churned and retained customers and was therefore selected for logistic regression modeling.

```
In [44]: import statsmodels.formula.api as smf
import numpy as np
```

```
In [45]: print(df[['churn', 'total_day_charge']].dtypes)
```

```
churn          Int64
total_day_charge  float64
dtype: object
```

```
In [46]: print(df[['churn', 'total_day_charge']].dtypes)
```

```
print(type(df['churn'].dtype))
print(type(df['total_day_charge'].dtype))
```

```
churn          Int64
total_day_charge  float64
dtype: object
<class 'pandas.core.arrays.integer.Int64Dtype'>
<class 'numpy.dtype'>
```

```
In [47]: # Clean churn completely
```

```

df['churn'] = pd.to_numeric(
    df['churn'],
    errors='coerce'
)

df['churn'] = (
    df['churn']
    .fillna(0)
)

df['churn'] = np.array(
    df['churn'],
    dtype='int64'
)

# Clean total_day_charge completely

df['total_day_charge'] = pd.to_numeric(
    df['total_day_charge'],
    errors='coerce'
)

df['total_day_charge'] = np.array(
    df['total_day_charge'],
    dtype='float64'
)

```

In [48]:

```

model = smf.logit(
    formula='churn ~ total_day_charge',
    data=df
).fit()

print(model.summary())

```

Optimization terminated successfully.
 Current function value: 0.392178
 Iterations 6

```

=====
Logit Regression Results
=====
Dep. Variable:          churn    No. Observations:          3333
Model:                Logit    Df Residuals:              3331
Method:                MLE     Df Model:                  1

```

```

Date:           Mon, 18 May 2026   Pseudo R-squ.:           0.05222
Time:           17:02:31          Log-Likelihood:          -1307.1
converged:      True              LL-Null:                  -1379.1
Covariance Type: nonrobust        LLR p-value:              3.491e-33
=====
               coef      std err          z      P>|z|      [0.025      0.975]
-----
Intercept      -3.9293      0.203     -19.373      0.000      -4.327      -3.532
total_day_charge  0.0663      0.006      11.561      0.000       0.055       0.078
=====

```

In [49]: `print(model.params)`

```

Intercept      -3.929328
total_day_charge  0.066304
dtype: float64

```

In [50]: `print(df['churn'].dtype)`

```
int64
```

In [51]: `print(model.params)`

```

Intercept      -3.929328
total_day_charge  0.066304
dtype: float64

```

In [52]: `model = smf.logit(
 formula='churn ~ total_day_charge',
 data=df
)`
`.fit()`

`print(model.summary())`

```

Optimization terminated successfully.
    Current function value: 0.392178
    Iterations 6

```

Logit Regression Results

```

=====
Dep. Variable:      churn   No. Observations:      3333
Model:              Logit   Df Residuals:          3331

```

```

Method:                MLE      Df Model:                1
Date:                  Mon, 18 May 2026      Pseudo R-squ.:          0.05222
Time:                  17:02:31      Log-Likelihood:         -1307.1
converged:              True      LL-Null:              -1379.1
Covariance Type:       nonrobust      LLR p-value:           3.491e-33
=====
                coef      std err          z      P>|z|      [0.025      0.975]
-----
Intercept        -3.9293      0.203     -19.373      0.000      -4.327      -3.532
total_day_charge   0.0663      0.006      11.561      0.000       0.055       0.078
=====

```

```

In [53]: df['predicted_churn_prob'] = model.predict(df)

df[['total_day_charge',
    'predicted_churn_prob']].head()

```

```

Out[53]:
   total_day_charge  predicted_churn_prob
0             31.94             0.140448
1             25.76             0.097851
2             51.54             0.374721
3             27.05             0.105665
4             36.69             0.182929

```

```

In [54]: df['predicted_churn_prob'] = model.predict(df)

df[
    [
        'total_day_charge',
        'predicted_churn_prob'
    ]
].head()

```

```

Out[54]:
   total_day_charge  predicted_churn_prob
0             31.94             0.140448

```

	total_day_charge	predicted_churn_prob
1	25.76	0.097851
2	51.54	0.374721
3	27.05	0.105665
4	36.69	0.182929

```
In [55]: risk_threshold = 0.35

high_risk_float = df[
    (df['predicted_churn_prob'] >= risk_threshold) &
    (df['churn'] == 0)
]

print(
    high_risk_float[
        [
            'phone_num',
            'city',
            'total_day_charge',
            'predicted_churn_prob'
        ]
    ].head(10)
)
```

	phone_num	city	total_day_charge	predicted_churn_prob
67	7802.0	Tallahassee	53.35	0.403235
99	3016.0	Jacksonville	51.88	0.380018
138	6436.0	Miami1	51.46	0.373479
139	7369.0	Tampa	52.68	0.392592
186	9999.0	Tallahassee	50.90	0.364833
673	5891.0	Jacksonville	52.21	0.385186
971	7120.0	Tallahassee	53.65	0.408030
1176	1994.0	Cape Coral	52.05	0.382677
1349	8583.0	Tallahassee	52.36	0.387544
1392	6784.0	Orlando2	52.77	0.394016

```
In [56]: plt.figure(figsize=(8,5))
```

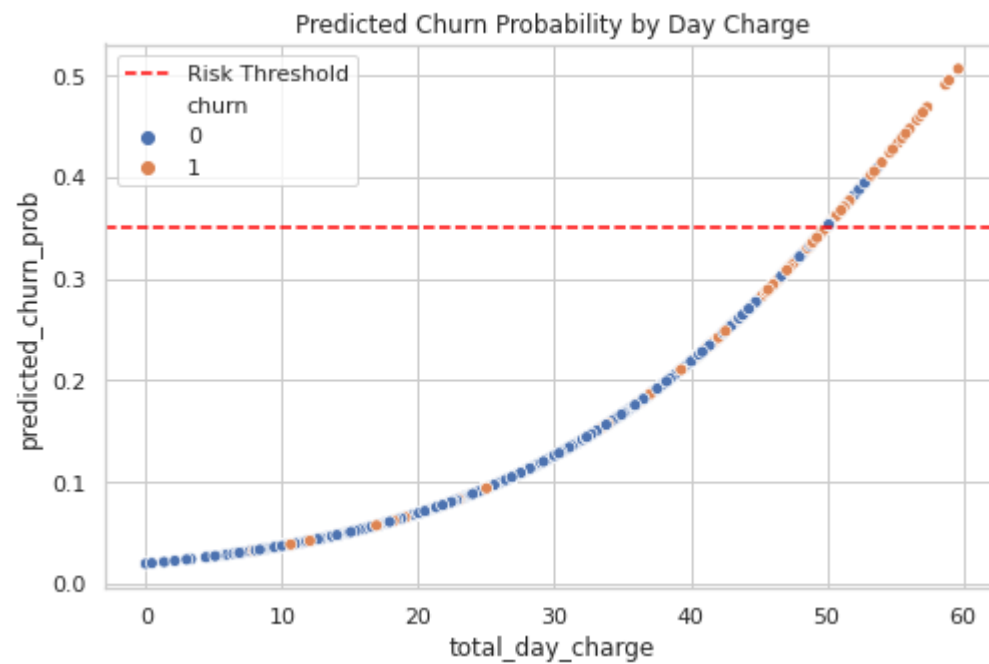
```
sns.scatterplot(
    x='total_day_charge',
    y='predicted_churn_prob',
    hue='churn',
    data=df
)

plt.axhline(
    y=risk_threshold,
    color='red',
    linestyle='--',
    label='Risk Threshold'
)

plt.title('Predicted Churn Probability by Day Charge')

plt.legend()

plt.show()
```



Congratulations: You have used logistic regression to predict which kinds of customers are likely to leave. Now you have finished analyzing the data and can focus on visualizing the results.

5) Visualizing the cities and other selected data series.

You will find examples from the following lessons helpful for this step:

- [Exploring Categories](#) (Chapter 2)
- [Visualizing Categories](#) (Chapter 2)
- [Combining Visualizations with matplotlib](#) (Chapter 2)
- [Customizing Visualizations with matplotlib](#) (Chapter 2)
- [Box Plots](#) (Chapter 3)

```
In [57]: overall_churn_rate = df['churn'].mean()

print(overall_churn_rate)
```

0.14422215586742312

```
In [58]: overall_churn_rate = df['churn'].mean()
```

```
In [59]: import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")

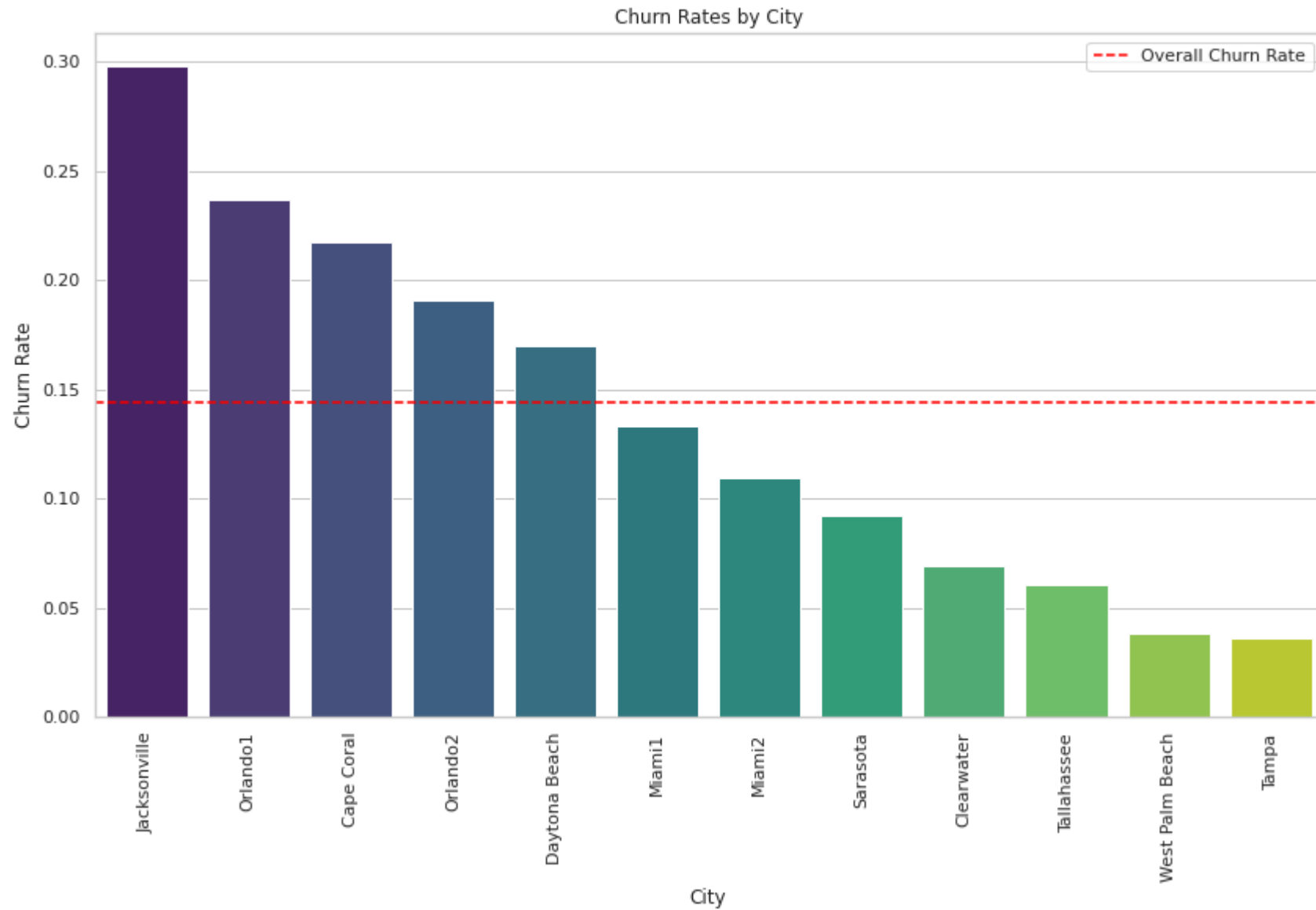
all_cities = city_churn

plt.figure(figsize=(14,8))

sns.barplot(
    x=all_cities.index,
    y=all_cities.values,
    palette='viridis'
)

plt.axhline(
```

```
y=overall_churn_rate,  
color='red',  
linestyle='--',  
label='Overall Churn Rate'  
)  
  
plt.title('Churn Rates by City')  
plt.xlabel('City')  
plt.ylabel('Churn Rate')  
  
plt.xticks(rotation=90)  
  
plt.legend()  
  
plt.show()
```



In [60]:

```
plt.figure(figsize=(8,5))

sns.lineplot(
    x=service_call_churn.index,
    y=service_call_churn.values,
    marker='o'
```

```
)

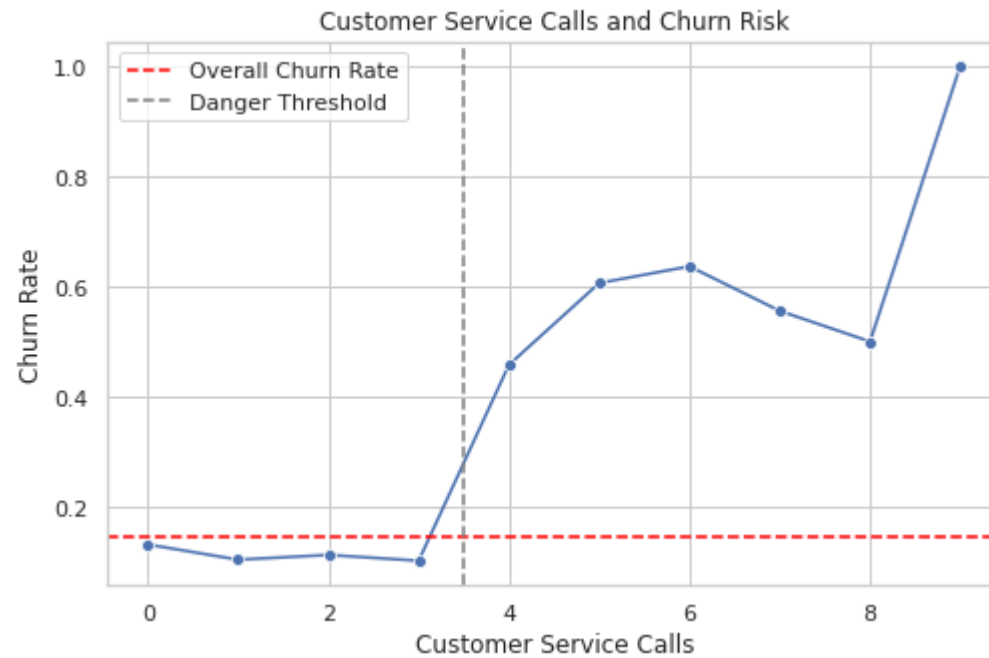
plt.axhline(
    y=overall_churn_rate,
    color='red',
    linestyle='--',
    label='Overall Churn Rate'
)

plt.axvline(
    x=3.5,
    color='gray',
    linestyle='--',
    label='Danger Threshold'
)

plt.title('Customer Service Calls and Churn Risk')
plt.xlabel('Customer Service Calls')
plt.ylabel('Churn Rate')

plt.legend()

plt.show()
```

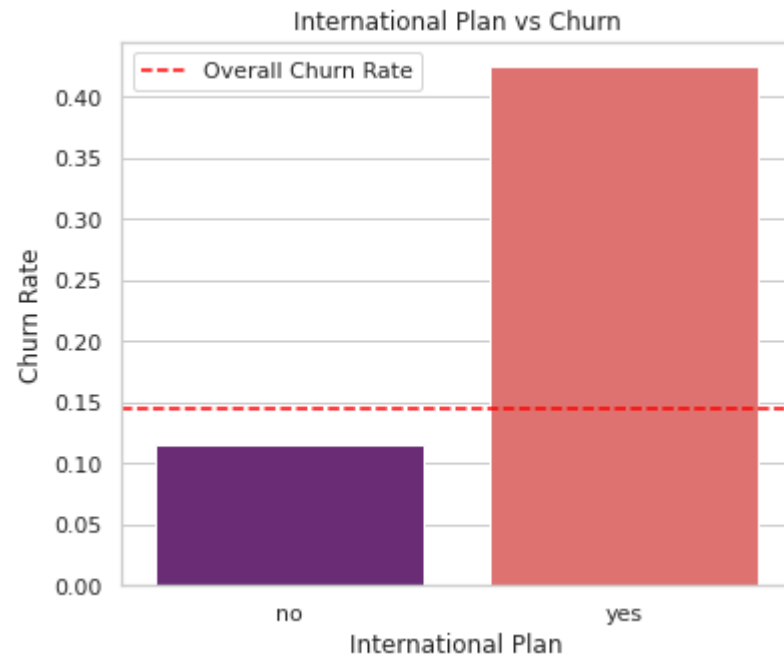


```
In [61]: plan_churn = (\n            df.groupby('international_plan')['churn']\n              .mean()\n          )\n\nplt.figure(figsize=(6,5))\n\nsns.barplot(\n    x=plan_churn.index,\n    y=plan_churn.values,\n    palette='magma'\n)\n\nplt.axhline(\n    y=overall_churn_rate,\n    color='red',\n    linestyle='--',\n    label='Overall Churn Rate'\n)\n\nplt.title('International Plan vs Churn')
```

```
plt.xlabel('International Plan')
plt.ylabel('Churn Rate')

plt.legend()

plt.show()
```

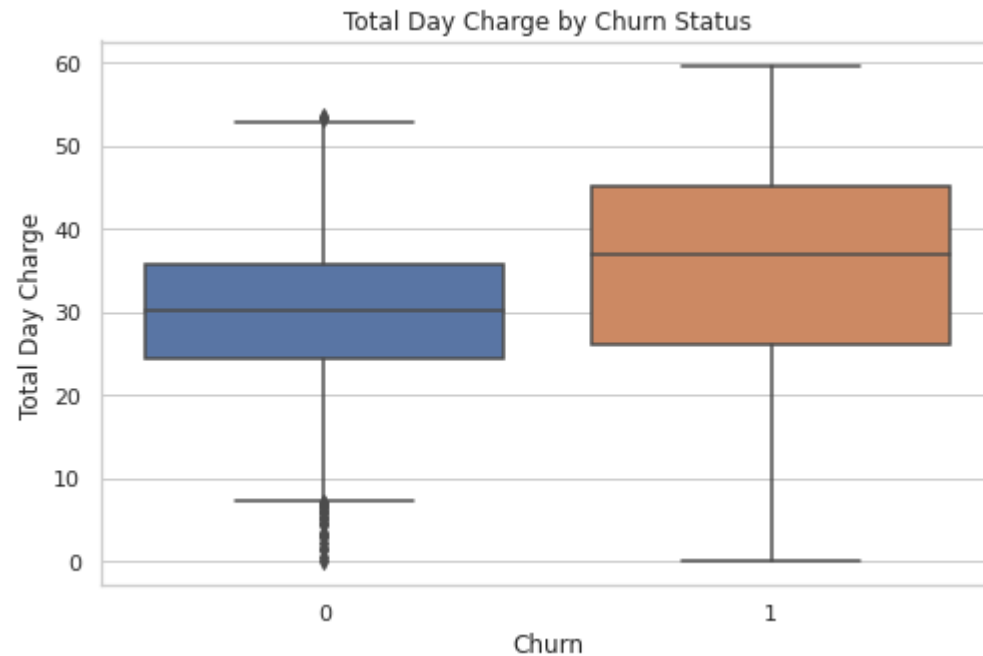


```
In [62]: plt.figure(figsize=(8,5))

sns.boxplot(
    x='churn',
    y='total_day_charge',
    data=df
)

plt.title('Total Day Charge by Churn Status')
plt.xlabel('Churn')
plt.ylabel('Total Day Charge')

plt.show()
```



6) Formulating a recommendation

We have two marketing campaigns: In the first, four cities with the highest customer churn are to be targeted with poster campaigns. In the second, customers who are ready to churn are to be approached individually in order to retain them as customers using special offers.

What recommendations would you give the telecommunications provider?

Write your observations, notes and recommendations from the data set in bullet points as comments with `#` in the following code cell.

```
In [63]: #Recommendation:  
#Focus retention marketing campaigns in high-churn cities.
```

```
In [64]: #Recommendation:  
#Review pricing and service quality for international plans.
```

```
In [65]: #Recommendation:  
#Create an early-warning retention system for customers exceeding the identified service-call threshold.
```

```
In [66]: #Recommendation:  
#Offer pricing reviews or customized usage plans for high-charge customers.
```

Conclusion:

This project analyzed customer churn behavior using demographic, service, and billing variables from a telecommunications company.

The analysis identified several important churn indicators, including geographic location, international plan subscription, customer service call frequency, and daytime charges.

Logistic regression and comparative visual analysis were used to identify customers with elevated churn probabilities and to generate targeted customer-retention lists.

The findings suggest that proactive retention strategies, improved customer support, and pricing adjustments may help reduce customer churn and improve long-term customer retention.

Congratulations: If you have made it this far then you have completed the last exercise in this training course! It wasn't always easy, but you have learned a lot. You can be proud of yourself for sticking with it to the end. Now you can consider yourself a data analyst.

Remember:

- Python can do all kinds of things. You will never run out of things to learn, so you should continue learning more skills as you progress. You will learn something new with each new data set.
- You have what it takes to be a data analyst! Python is now one of the tools you have at your disposal.

Do you have any questions about this exercise? Look in the [forum](#) to see if they have already been discussed.

Found a mistake? Contact Support at support@stackfuel.com.
